Erise-AI 企业级开发设计文档（V1）

1. 文档说明

1.1 文档目的

本文档用于指导 **Erise-AI 私有垂直化项目知识库管理系统** 的产品设计、系统设计、研发实施、接口协同、数据库建设、测试验收与后续运维，作为前后端、AI 微服务、测试、运维统一遵循的基线文档。

1.2 适用范围

适用于以下研发对象： - 前端应用：`erise-ai-ui` - 后端主体：`erise-ai-backend` - AI 微服务：`erise-ai-cloud` - 管理后台：建议复用同一前端工程中的管理端路由与布局，必要时可拆分独立管理端 - 数据层：MySQL、Redis、对象存储、向量检索组件

1.3 项目定位

Erise-AI 是一个面向 **个人用户或轻量团队** 的私有项目知识库管理系统，围绕： - 项目 - 文件 - 文档 - 搜索 - AI 助手

建立统一的知识沉淀、结构化管理、内容编辑与可信检索空间。

系统目标并不只是“上传文件”，而是把“项目内知识”转化为可持续复用、可被 AI 安全调用、可检索、可编辑、可追溯、可协作的资产。

---

2. 建设目标

2.1 业务目标

1. 支持个人或轻量团队按“项目”组织知识资产。
2. 支持多类型文件上传、预览、下载、标签分类与历史浏览。
3. 支持在线文档编辑，具备常用知识库编辑能力：
4. 富文本
5. 代码块
6. 图片插入/编辑
7. PDF 预览
8. 常见附件引用
9. 支持全文搜索、标签搜索、项目内搜索、文件内容搜索。
10. 构建基于知识库内容的 AI 助手，回答严格基于知识库上下文。
11. 支持管理员后台，对用户、项目、文件、存储、任务、审计进行管理。

2.2 技术目标

1. 前后端分离，模块清晰，边界明确。
2. 主体业务服务与 AI 微服务解耦。
3. 接口规范、异常规范、权限规范统一。
4. 支持后续扩展：团队协作、版本管理、向量检索、外部工具接入。

5. 面向生产可演进，避免“单体混乱堆功能”。

2.3 非功能目标

1. 安全：鉴权、权限控制、审计、文件访问控制、Prompt 注入防护。
2. 性能：大文件上传、分片任务、异步解析、缓存加速、搜索响应优化。
3. 可维护：统一命名、统一 DTO/VO、统一响应结构、统一日志链路。
4. 可观测：日志、审计、接口追踪、任务状态、AI 调用记录。
5. 可扩展：搜索组件、向量库、对象存储、模型服务可替换。

---

3. 目标用户与权限模型

3.1 用户角色

系统初期定义两类角色： - **普通用户**：管理自己的项目、文件、文档、标签、历史记录，并使用 AI 助手 - **管理员**：管理全站用户、全站项目统计、文件审计、任务状态、系统参数、模型接入配置等

3.2 权限原则

采用 **RBAC + 资源归属校验** 双层控制： - 第一层：角色权限（管理员 / 普通用户） - 第二层：资源归属权限（用户是否拥有该项目、文件、文档）

3.3 权限控制要求

- 普通用户仅可访问自己拥有或被授权的资源
- 管理员拥有后台管理权限，但应保留审计记录
- AI 工具调用必须复用业务权限体系，不能绕过业务权限
- 所有下载、删除、恢复、分享、导出操作必须记录审计日志

---

4. 总体架构设计

4.1 架构原则

1. **前后端分离**
2. **主体业务与 AI 能力解耦**
3. **编辑、文件、搜索、AI、任务分层治理**
4. **在线事务与异步任务拆分**
5. **统一网关规范/统一 API 规范/统一安全策略**

4.2 系统逻辑架构

```
[ erise-ai-ui 前端 ]
        |
        | HTTP / WebSocket
        v
[ erise-ai-backend 主体后端 ]
    |- auth 认证中心
    |- user 用户中心
```

```
    |- project 项目中心
    |- file 文件中心
    |- document 文档中心
    |- tag 标签中心
    |- search 检索中心
    |- task 任务中心
    |- history 历史中心
    |- admin 管理中心
    |- editor 协同编辑/WebSocket
    |
    |- MySQL
    |- Redis
    |- Object Storage (MinIO/OSS/S3)
    |
    |- 解析任务 / 索引任务 / 缩略图任务 / 文本抽取任务
    v
[ erise-ai-cloud AI 微服务 ]
    |- Spring AI
    |- OpenAI 接入
    |- RAG 编排
    |- Tool Calling / Function Calling
    |- MCP Client / MCP Gateway
    |- SQL / 文件 / 检索工具代理
    |- 引用与答案约束
    |
    |- 向量索引 / Embedding / 检索重排
```

4.3 推荐部署拆分

- `erise-ai-ui`：前端静态站点或 Node 构建产物
- `erise-ai-backend`：核心业务服务
- `erise-ai-cloud`：AI 微服务
- `mysql`：业务数据库
- `redis`：缓存、会话、任务状态
- `minio`：对象存储
- `vector-store`：向量检索层（可选 PGVector / Elasticsearch / Milvus）

4.4 为什么要拆分 AI 微服务

AI 相关能力变化快、依赖重、调用成本高、故障特征不同，不建议与主体业务完全耦合。拆分后具备： - 独立扩缩容 - 独立限流与熔断 - 独立模型配置 - 独立审计与成本统计 - 便于替换模型厂商、Embedding 模型与向量检索实现

4.5 部署方式建议（补充）

Erise-AI 推荐采用 **容器化部署**，即以 **Docker 镜像 + Docker Compose** 作为本地开发、联调、测试与轻量生产环境的主部署方式；在用户规模、并发量或运维复杂度提升后，再平滑演进到 **Kubernetes**。

4.5.1 推荐部署分层

- **本地开发环境**：前端可本机 Node.js 运行，后端/数据库/Redis/MinIO/向量库建议 Docker Compose 启动
- **联调/测试环境**：前端、主体后端、AI 微服务、MySQL、Redis、MinIO、向量库统一走 Docker Compose
- **生产环境（轻量）**：建议整套容器化部署，Nginx + UI 静态资源容器 + 后端容器 + AI 微服务容器 + MySQL + Redis + MinIO
- **生产环境（进阶）**：当需要弹性扩缩容、多副本、高可用和更完善运维治理时，再迁移到 Kubernetes

4.5.2 为什么优先 Docker / Docker Compose

1. 更适合当前 Erise-AI 的 **前后端分离 + 多基础组件** 架构
2. MySQL、Redis、MinIO、向量库、后端、AI 微服务可以统一编排，降低本地与测试环境差异
3. 对个人开发者或轻量团队更友好，部署门槛低，联调效率高
4. 更便于后续在同一镜像基础上迁移到云主机、私有服务器或 Kubernetes

4.5.3 Erise-AI 推荐容器清单

- `erise-ai-ui`：前端静态资源容器（Nginx 承载）
- `erise-ai-backend`：主体后端容器
- `erise-ai-cloud`：AI 微服务容器
- `mysql`：结构化数据存储
- `redis`：缓存/会话/任务状态
- `minio`：对象存储
- `vector-store`：向量检索（PGVector / Elasticsearch / Milvus 三选一）
- `nginx`：统一入口反向代理

4.5.4 本地开发建议

- 前端支持两种模式：
- **模式 A：本机运行** `npm run dev`
- **模式 B：Docker 容器运行** 适合联调与复现
- 主体后端与 AI 微服务建议容器化运行，减少 JDK、依赖、环境变量差异问题
- MySQL、Redis、MinIO 建议始终容器化，便于重建与迁移

4.5.5 生产部署建议

- 初期：`Docker Compose + Nginx + 定时备份 + 日志采集`
- 中期：将 MySQL、Redis、对象存储托管化或独立部署
- 后期：迁移 Kubernetes，拆分多副本、弹性扩容、灰度发布与监控告警

5. 技术栈建议

5.1 前端技术栈

目录：`erise-ai-ui`

建议采用： - Node.js 24 LTS - Vue 3 - Vite - Vue Router - Axios - Element Plus - @element-plus/icons-vue - Pinia（建议补充） - TypeScript（强烈建议引入） - UnoCSS 或 Tailwind（二选一，非必须） - 富文本/文档编辑器：Tiptap / Milkdown / Editor.js（三选一，推荐 Tiptap） - PDF 预览：pdf.js - 代码编辑：Monaco Editor 或 CodeMirror - 上传组件：Element Plus Upload + 自定义分片逻辑

5.2 后端主体技术栈

目录：`erise-ai-backend`

建议采用： - Temurin OpenJDK 21 - Spring Boot - Spring MVC - Spring Security - MyBatis-Plus - Maven - MySQL 8.x - Redis - Sa-Token 或 JWT + Spring Security（二选一，推荐统一使用 Spring Security + JWT/Refresh Token） - WebSocket - MapStruct（建议补充） - Hutool（可选） - Lombok - Spring Validation - Knife4j / OpenAPI

5.3 AI 微服务技术栈

目录：`erise-ai-cloud`

建议采用： - Spring Boot - Spring Cloud（仅用于微服务治理时引入，轻量单机版可先不强依赖注册中心） - Spring AI - OpenAI 兼容模型接入 - Embedding 模型接入 - 向量存储集成 - RAG 检索编排 - Function Calling / Tool Calling - MCP Client / Tool Adapter

5.4 存储与搜索建议

- 结构化数据：MySQL
- 缓存与任务状态：Redis
- 文件二进制：MinIO / S3
- 全文搜索：MySQL FullText（初期）或 Elasticsearch（进阶）
- 向量检索：PGVector / Elasticsearch Vector / Milvus（三选一）

5.5 初期推荐方案（兼顾开发成本）

为了尽快落地，推荐 V1： - MySQL + Redis + MinIO - 全文搜索先走 MySQL/ES 二选一 - 向量库优先 PGVector 或 Elasticsearch Vector - 前端使用 Vue3 + Vite + TS + Element Plus + Tiptap

---

6. 目录结构设计

6.1 前端目录结构（推荐）

```
erise-ai-ui/
├── public/
├── src/
│   ├── api/
│   │   ├── auth.ts
│   │   ├── user.ts
│   │   ├── project.ts
│   │   ├── file.ts
│   │   ├── document.ts
│   │   ├── search.ts
│   │   ├── task.ts
│   │   ├── admin.ts
│   │   └── ai.ts
│   ├── assets/
│   ├── components/
```

```
│   │   ├── common/
│   │   ├── layout/
│   │   ├── editor/
│   │   ├── file/
│   │   ├── project/
│   │   ├── search/
│   │   └── ai/
│   ├── constants/
│   ├── composables/
│   ├── layouts/
│   ├── router/
│   │   ├── index.ts
│   │   ├── modules/
│   ├── stores/
│   │   ├── auth.ts
│   │   ├── user.ts
│   │   ├── project.ts
│   │   └── app.ts
│   ├── types/
│   ├── utils/
│   ├── views/
│   │   ├── auth/
│   │   ├── workspace/
│   │   ├── project/
│   │   ├── file/
│   │   ├── document/
│   │   ├── search/
│   │   ├── admin/
│   │   └── ai/
│   ├── App.vue
│   └── main.ts
├── .env.development
├── .env.production
├── vite.config.ts
├── tsconfig.json
└── package.json
```

6.2 后端主体目录结构（推荐）

```
erise-ai-backend/
├── src/main/java/com/erise/ai/
│   ├── common/
│   │   ├── api/
│   │   ├── config/
│   │   ├── constant/
│   │   ├── enums/
│   │   ├── exception/
│   │   ├── security/
│   │   ├── util/
│   │   └── web/
```

```
│   ├── modules/
│   │   ├── auth/
│   │   │   ├── controller/
│   │   │   ├── service/
│   │   │   ├── dto/
│   │   │   ├── vo/
│   │   │   └── mapper/
│   │   ├── user/
│   │   ├── project/
│   │   ├── file/
│   │   ├── document/
│   │   ├── tag/
│   │   ├── search/
│   │   ├── task/
│   │   ├── history/
│   │   ├── admin/
│   │   └── editor/
│   ├── integration/
│   │   ├── storage/
│   │   ├── ai/
│   │   ├── search/
│   │   └── websocket/
│   └── EriseAiBackendApplication.java
├── src/main/resources/
│   ├── mapper/
│   ├── application.yml
│   ├── application-dev.yml
│   ├── application-prod.yml
│   └── db/migration/
└── pom.xml
```

6.3 AI 微服务目录结构（推荐）

```
erise-ai-cloud/
├── src/main/java/com/erise/ai/cloud/
│   ├── common/
│   ├── config/
│   ├── controller/
│   ├── service/
│   ├── rag/
│   │   ├── ingest/
│   │   ├── retrieve/
│   │   ├── rerank/
│   │   ├── cite/
│   │   └── guard/
│   ├── tools/
│   │   ├── file/
│   │   ├── sql/
│   │   ├── project/
│   │   └── search/
```

```
│   ├── mcp/
│   ├── provider/
│   └── EriseAiCloudApplication.java
└── pom.xml
```

---

7. 模块职责划分

7.1 核心业务模块

7.1.1 auth 认证模块

职责： - 注册 / 登录 / 登出 - Token 签发与刷新 - 密码加密与校验 - 验证码 / 邮箱验证码（如需要） - 账号状态校验（禁用、锁定）

7.1.2 user 用户模块

职责： - 用户资料 - 头像、昵称、邮箱等基本信息 - 用户配置项 - 个人 AI 参数偏好（模型、输出风格、历史保留等）

7.1.3 project 项目模块

职责： - 项目创建、修改、归档、删除 - 项目封面、说明、标签 - 项目统计 - 项目与文件、文档、聊天会话关联

7.1.4 file 文件模块

职责： - 文件上传、秒传、分片、合并 - 文件元数据管理 - 文件预览信息 - 文件下载控制 - 文件版本/去重（后续可扩展） - 文件解析任务投递

7.1.5 document 文档模块

职责： - 在线文档创建、编辑、发布、保存草稿 - 文档内容块结构化存储 - 图片、代码块、附件引用 - 文档版本历史 - 文档预览

7.1.6 tag 标签模块

职责： - 文件/项目/文档标签统一管理 - 标签搜索 - 标签统计

7.1.7 search 搜索模块

职责： - 项目内搜索 - 文件名搜索 - 文档内容搜索 - 标签搜索 - 混合检索入口（关键词 + 语义）

7.1.8 task 任务模块

职责： - 文件解析任务 - OCR / 文本抽取任务 - 缩略图生成任务 - 向量化任务 - 索引构建任务 - 失败重试与状态跟踪

7.1.9 history 历史模块

职责： - 最近访问 - 最近编辑 - 下载历史 - 搜索历史 - AI 会话历史

7.1.10 admin 管理模块

职责： - 用户管理 - 角色与权限管理（后续扩展） - 项目审计 - 文件审计 - 存储统计 - AI 配置管理 - 系统字典与参数管理

7.1.11 editor 协同编辑模块

职责： - 文档实时保存 - WebSocket 推送 - 编辑锁/会话状态 - 预留多人协同能力

7.2 AI 微服务模块

7.2.1 rag 数据接入与检索模块

职责： - 文件解析结果接入 - 文本分段与清洗 - Embedding 生成 - 向量索引写入 - 召回、重排、引用包装

7.2.2 tools 工具调用模块

职责： - 按权限读取文件 - 查询项目信息 - 执行只读 SQL 查询 - 调用搜索接口 - 操作文档或文件（必须走权限控制）

7.2.3 mcp 模块

职责： - 对接 MCP Server - 管理工具注册 - 将外部工具能力纳入 AI 调用链 - 做工具白名单与参数校验

7.2.4 provider 模型接入模块

职责： - OpenAI / 兼容供应商接入 - Embedding 模型接入 - 重排序模型接入 - 模型参数配置化

---

8. 分层与编码规范

8.1 后端分层强制规范

必须遵循： - **Controller**：只负责接收参数、校验基础格式、调用 Service、返回统一响应 - **Service**：负责业务流程、权限校验、事务控制、编排多模块逻辑 - **Mapper**：只负责数据库访问，不写业务流程 - **Entity**：仅用于数据库映射，不直接返回前端 - **DTO**：接收入参 - **VO**：返回前端视图对象 - **Convert/Assembler**：负责 DTO/Entity/VO 转换

8.2 禁止事项

- Controller 中写复杂业务编排
- Controller 中直接操作 Mapper
- Entity 直接对外返回
- Service 之间相互循环依赖
- AI 工具直接绕过业务 Service 访问底层数据库

8.3 推荐命名

- Controller：`ProjectController`
- Service：`ProjectService`
- ServiceImpl：`ProjectServiceImpl`

- Mapper：`ProjectMapper`
- Entity：`ProjectEntity`
- DTO：`ProjectCreateRequest`
- VO：`ProjectDetailVO`

8.4 事务规范

- 单模块写操作由对应 Service 控制事务
- 跨模块写操作由编排 Service 控制事务
- 文件上传成功但数据库写失败，需要做补偿或标记孤儿对象清理
- AI 调用链默认不参与长事务

---

9. API 统一规范

9.1 接口风格

统一使用 RESTful 风格 + 统一响应体。

9.2 URL 前缀规范

推荐统一前缀：

```
/api/v1
```

模块路径示例：- `/api/v1/auth/login` - `/api/v1/auth/logout` - `/api/v1/users/me` - `/api/v1/projects` - `/api/v1/projects/{id}` - `/api/v1/files/upload` - `/api/v1/files/{id}/download` - `/api/v1/documents/{id}` - `/api/v1/search` - `/api/v1/tasks/{id}` - `/api/v1/admin/users` - `/api/v1/ai/chat`

9.3 响应体规范

```
{
 "code": 0,
 "message": "success",
 "data": {}
}
```

分页统一结构：

```
{
 "code": 0,
 "message": "success",
 "data": {
  "records": [],
  "pageNum": 1,
  "pageSize": 20,
  "total": 100,
  "totalPages": 5
```

```
  }
}
```

9.4 错误码规范

建议错误码分段： - `0`：成功 - `400xxx`：参数错误 - `401xxx`：认证失败 - `403xxx`：权限不足 - `404xxx`：资源不存在 - `409xxx`：状态冲突 - `500xxx`：系统异常 - `510xxx`：文件处理异常 - `520xxx`：AI 调用异常 - `530xxx`：搜索检索异常

9.5 鉴权规范

- 登录成功返回 Access Token + Refresh Token
- 所有业务接口通过 Bearer Token 鉴权
- 文件下载可采用短期签名 URL 或受控下载接口
- WebSocket 连接必须校验 Token

9.6 幂等与安全

- 上传接口需要幂等支持
- 删除接口建议软删除
- 批量操作必须校验资源归属
- 管理员操作必须记录操作日志

---

10. 数据库设计原则

10.1 总体原则

1. 统一表前缀：建议 `ea_`
2. 每张核心表具备：
3. `id`
4. `created_by`
5. `created_at`
6. `updated_by`
7. `updated_at`
8. `deleted`
9. 字段命名统一使用下划线命名法
10. 不在数据库中存业务大 JSON 作为主结构，结构化优先
11. 文档内容、文件元信息、索引状态、AI 会话拆表存储

10.2 核心数据域

10.2.1 用户域

- `ea_user`
- `ea_user_profile`
- `ea_user_setting`
- `ea_user_login_log`

10.2.2 项目域

- `ea_project`
- `ea_project_member`（后续团队版）
- `ea_project_tag_rel`

10.2.3 文件域

- `ea_file`
- `ea_file_version`
- `ea_file_tag_rel`
- `ea_file_parse_task`
- `ea_file_chunk`
- `ea_file_preview`

10.2.4 文档域

- `ea_document`
- `ea_document_content`
- `ea_document_version`
- `ea_document_tag_rel`

10.2.5 搜索与知识块域

- `ea_knowledge_chunk`
- `ea_knowledge_index_job`
- `ea_search_history`

10.2.6 AI 域

- `ea_ai_session`
- `ea_ai_message`
- `ea_ai_citation`
- `ea_ai_tool_call_log`
- `ea_ai_model_config`

10.2.7 管理与审计域

- `ea_audit_log`
- `ea_operation_log`
- `ea_system_config`

10.3 文件表建议字段

`ea_file` 建议包括： - id - user_id - project_id - file_name - file_ext - mime_type - file_size - storage_provider - storage_bucket - storage_key - checksum_md5 / sha256 - parse_status - preview_status - index_status - source_type - deleted - created_at / updated_at

10.4 文档表建议字段

`ea_document` 建议包括： - id - user_id - project_id - title - summary - cover_url - doc_status（draft/published/archived） - latest_version_no - content_type - deleted - created_at / updated_at

10.5 软删除原则

核心业务表建议使用逻辑删除，但文件物理对象删除采用“延迟清理任务”处理，避免误删。

---

11. 文件与文档能力设计

11.1 文件能力范围

系统需要支持： - 图片 - PDF - Word - Excel - Markdown - TXT - 代码文件 - 压缩包（仅下载，不在线编辑）

11.2 文件处理流程

1. 前端请求上传凭证 / 初始化上传
2. 上传二进制到对象存储
3. 后端写文件元数据
4. 投递异步任务：
5. 预览转换
6. 文本抽取
7. OCR（可选）
8. 向量化索引
9. 前端轮询或订阅任务状态

11.3 文档编辑能力

V1 建议采用块式或富文本混合结构，支持： - 标题/段落/引用 - 图片块 - 代码块 - 表格 - 文件引用 - PDF 嵌入预览 - Markdown 导入/导出（可选）

11.4 文档存储方式建议

文档不要只存整段 HTML。建议采用： - 主表保存元数据 - 内容表保存 JSON 结构或编辑器 schema - 发布态可缓存 HTML/纯文本快照

这样更利于： - 后续版本对比 - 搜索抽取 - 知识块切分 - AI 引用定位

11.5 PDF 预览

- 前端使用 pdf.js
- 后端只提供受控文件流或签名 URL
- PDF 文本抽取结果进入搜索与 RAG 索引

---

12. 搜索系统设计

12.1 搜索目标

支持用户快速找到： - 项目 - 文件 - 文档 - 标签 - 文档片段 - AI 可引用内容

12.2 搜索类型

1. **结构化搜索**：按项目、文件类型、标签、时间筛选
2. **关键词搜索**：标题、文件名、正文内容
3. **语义搜索**：根据 Embedding 检索语义相似内容
4. **混合检索**：关键词召回 + 向量召回 + 重排

12.3 推荐演进路线

- V1：MySQL/ES 关键词搜索
- V2：接入向量检索
- V3：混合检索 + rerank + 引用片段高亮

12.4 搜索索引对象

- 文档标题
- 文档正文块
- 文件抽取文本
- 项目描述
- 标签
- OCR 内容（可选）

12.5 搜索结果展示要求

- 类型标识（文件/文档/项目）
- 所属项目
- 片段摘要
- 命中高亮
- 可直接跳转

---

13. AI 助手与 RAG 设计

13.1 AI 产品定位

AI 不是一个泛聊天入口，而是一个 **专属知识库助手**，核心能力包括： - 基于知识库内容回答问题 - 总结项目材料 - 解释文档内容 - 生成草稿 - 辅助整理标签/摘要 - 基于权限执行受控工具操作

13.2 回答原则

AI 回答必须遵循： 1. **严格基于知识库内容作答** 2. **有引用再回答，无引用则拒答或降级回答** 3. **不允许绕过权限读取非授权内容** 4. **所有工具调用必须经过白名单与参数校验**

13.3 RAG 流程

```
用户提问
  -> 权限过滤
  -> 查询改写
  -> 混合检索
  -> 重排序
  -> 上下文组装
```

```
 -> 引用注入
 -> 大模型生成
 -> 输出答案 + 引用来源
```

13.4 知识入库流程

```
文件/文档创建或更新
 -> 文本提取
 -> 内容清洗
 -> 分块切片
 -> 生成 embedding
 -> 写入向量索引
 -> 写入引用元数据
```

13.5 分块策略建议

- 按自然段/标题层级优先切分
- 控制块大小，保留上下文重叠
- 记录来源元信息：
- project_id
- source_type
- source_id
- chunk_index
- title/path
- page_no（如 PDF）

13.6 AI 输出要求

AI 返回结构建议： - answer - citations - used_tools - confidence - refused_reason（如无依据）

13.7 引用规范

引用至少包含： - 来源类型 - 文件/文档标题 - 所属项目 - 片段位置（页码/段落/块号）

13.8 Prompt 注入与越权防护

必须增加： - 文件内容中恶意提示过滤 - 工具调用白名单 - SQL 工具只读限制 - 工具参数模式校验 - 输出前二次校验引用是否真实存在

---

14. Function Calling 与 MCP 工具体系

14.1 工具设计原则

AI 工具不是直接把数据库暴露给模型，而是把“**受控能力**”暴露给模型。

14.2 推荐工具分类

1. 只读查询工具

2. 查询项目列表
3. 查询文件详情
4. 查询文档内容摘要
5. 搜索知识库
6. 受控操作工具
7. 创建文档草稿
8. 给文件添加标签
9. 移动文件到项目
10. 启动文档总结任务
11. 管理工具（仅管理员）
12. 查询系统统计
13. 查询任务失败情况
14. 查看审计日志摘要

14.3 SQL 工具规范

如果提供 SQL 查询能力，必须限制为： - 只读 SQL - 仅允许访问白名单视图/白名单表 - 结果集大小限制 - 审计记录完整 - 不允许 DDL / DML / 多语句执行

14.4 MCP 接入建议

MCP 适合作为扩展工具协议层，用于： - 文件系统访问 - 数据库只读查询 - 外部知识源接入 - Dev 工具接入

但业务核心工具优先走内部 Tool API，不建议一开始把核心业务完全建立在外部 MCP Server 上。

14.5 工具调用安全链路

```
模型提出工具调用
 -> Tool Dispatcher
 -> 权限校验
 -> 参数校验
 -> 业务 Service 调用
 -> 审计日志记录
 -> 结果脱敏
 -> 返回模型
```

---

15. WebSocket 与实时能力设计

15.1 使用场景

- 文档自动保存状态通知
- AI 流式回复
- 任务进度通知
- 上传解析完成通知
- 协同编辑在线状态

15.2 通道拆分建议

- `/ws/editor`
- `/ws/task`
- `/ws/ai`
- `/ws/notify`

15.3 设计原则

- WebSocket 连接建立前必须鉴权
- 消息体定义统一 schema
- 心跳与断线重连机制完善
- 服务端推送不承载复杂业务事务

---

16. 安全设计

16.1 认证安全

- 密码强哈希存储（BCrypt/Argon2）
- 登录限流
- 验证码/风控
- Token 失效与刷新机制
- 异地登录策略（可选）

16.2 资源安全

- 文件访问必须校验资源归属
- 下载地址采用短时签名或网关受控转发
- 删除操作支持回收站

16.3 AI 安全

- Prompt 注入检测
- 工具调用白名单
- SQL 工具只读
- 输出敏感信息脱敏
- AI 调用审计

16.4 接口安全

- 统一异常拦截
- 参数校验
- CORS 规范
- XSS/CSRF/注入防护
- 文件上传类型与大小限制

16.5 审计要求

以下行为必须审计： - 登录/登出 - 文件上传/下载/删除 - 文档发布/删除 - 管理员操作 - AI 工具调用 - SQL 查询工具调用

---

17. 日志、监控与运维

17.1 日志规范

建议日志分层： - 应用日志 - 审计日志 - 任务日志 - AI 调用日志 - 安全日志

17.2 日志字段建议

- traceId
- userId
- projectId
- requestUri
- method
- costMs
- resultCode
- toolName（如 AI 工具）

17.3 监控指标

- 接口成功率/耗时
- 文件上传成功率
- 解析任务成功率
- 搜索响应时间
- AI 调用耗时/失败率/Token 成本
- WebSocket 在线数

17.4 配置管理

- 环境变量 + yml 分环境管理
- 模型 key 不落库明文
- 存储密钥、数据库密钥统一加密管理

---

18. 前端页面规划

18.1 普通用户端

1. 登录页
2. 工作台首页
3. 项目列表页
4. 项目详情页
5. 文件管理页
6. 文档编辑页
7. 文档详情页
8. 搜索中心页
9. AI 助手页
10. 最近历史页
11. 个人设置页

18.2 管理后台

1. 管理首页/仪表盘
2. 用户管理
3. 项目审计
4. 文件审计
5. 任务管理
6. AI 配置管理
7. 系统配置管理
8. 操作日志/审计日志

18.3 UI 原则

- 用户端强调内容与效率
- 管理端强调表格、筛选、统计、审计
- 统一设计语言、色彩、间距与空状态组件

---

19. 核心接口清单（建议）

19.1 认证接口

- `POST /api/v1/auth/register`
- `POST /api/v1/auth/login`
- `POST /api/v1/auth/logout`
- `POST /api/v1/auth/refresh`
- `GET /api/v1/auth/captcha`（如启用）

19.2 用户接口

- `GET /api/v1/users/me`
- `PUT /api/v1/users/me`
- `PUT /api/v1/users/password`

19.3 项目接口

- `GET /api/v1/projects`
- `POST /api/v1/projects`
- `GET /api/v1/projects/{id}`
- `PUT /api/v1/projects/{id}`
- `DELETE /api/v1/projects/{id}`

19.4 文件接口

- `POST /api/v1/files/init-upload`
- `POST /api/v1/files/upload`
- `POST /api/v1/files/complete-upload`
- `GET /api/v1/files/{id}`
- `GET /api/v1/files/{id}/preview`
- `GET /api/v1/files/{id}/download`
- `DELETE /api/v1/files/{id}`

19.5 文档接口

- `POST /api/v1/documents`
- `GET /api/v1/documents/{id}`
- `PUT /api/v1/documents/{id}`
- `POST /api/v1/documents/{id}/publish`
- `GET /api/v1/documents/{id}/versions`

19.6 搜索接口

- `GET /api/v1/search`
- `GET /api/v1/search/suggest`
- `GET /api/v1/search/history`

19.7 AI 接口

- `POST /api/v1/ai/chat`
- `POST /api/v1/ai/chat/stream`
- `GET /api/v1/ai/sessions`
- `GET /api/v1/ai/sessions/{id}`

19.8 管理接口

- `GET /api/v1/admin/users`
- `GET /api/v1/admin/projects`
- `GET /api/v1/admin/files`
- `GET /api/v1/admin/tasks`
- `GET /api/v1/admin/audit-logs`

---

20. 典型业务流程

20.1 上传文件并进入知识库

1. 用户进入项目
2. 前端初始化上传
3. 文件上传到对象存储
4. 写入文件表
5. 创建解析任务
6. 提取文本与预览信息
7. 建立搜索索引与向量索引
8. 前端展示“可搜索 / 可 AI 引用”状态

20.2 创建在线文档

1. 用户创建文档
2. 保存文档元数据
3. 自动保存内容快照
4. 发布后进入搜索与知识索引
5. AI 可引用该文档片段

20.3 AI 问答

1. 用户在项目内提问
2. 后端传递用户、项目、会话上下文
3. AI 微服务做权限过滤
4. 检索项目相关知识块
5. 重排与引用组装
6. 调用模型生成答案
7. 返回答案、引用来源、可选工具结果

20.4 AI 调用工具给文档打标签

1. 用户提出意图
2. 模型选择“标签工具”
3. 工具层校验用户是否可编辑该资源
4. 调用主体后端标签服务
5. 返回执行结果并记录审计

---

21. 测试与验收标准

21.1 测试层次

- 单元测试
- 接口测试
- 集成测试
- 前后端联调测试
- 安全测试
- 性能测试
- AI 可信性测试

21.2 基础验收项

1. 登录、鉴权、权限隔离正确
2. 文件上传、预览、下载正常
3. 文档编辑、保存、版本历史正常
4. 搜索可搜到文件与文档内容
5. AI 回答能给出引用来源
6. AI 不返回无权限内容
7. 管理员后台可查询审计日志

21.3 AI 验收项

- 问题命中知识库时可给出来源引用
- 未命中时能明确说明依据不足
- 工具调用不会越权
- SQL 工具不会执行写操作

---

22. 开发阶段建议

22.1 Phase 1：基础骨架

- 用户认证
- 项目管理
- 文件上传/下载
- 文档基础编辑
- 基础搜索
- 管理后台骨架

22.2 Phase 2：知识化能力

- 文件解析
- 文本抽取
- 文档内容索引
- 搜索增强
- 历史记录

22.3 Phase 3：AI 能力

- Spring AI 接入
- RAG 流程
- 引用输出
- AI 会话历史
- 基础工具调用

22.4 Phase 4：高级能力

- MCP 扩展
- SQL 工具
- 更强的混合检索
- 协同编辑增强
- 管理后台增强

---

23. V1 推荐落地边界

为防止项目一开始过重，建议 V1 先做以下闭环： - 用户登录注册 - 项目 CRUD - 文件上传/预览/下载 - 在线文档编辑（基础版） - 关键词搜索 - 基于项目知识的 AI 问答（带引用） - 管理后台基础统计与用户管理

暂缓内容： - 真正多人协同编辑 - 复杂权限共享体系 - 高级工作流编排 - 太复杂的 SQL 智能分析能力 - 多模型多租户计费体系

---

24. 最终建议与结论

24.1 关键设计结论

1. **主体业务后端与 AI 微服务必须拆分**

2. **文件、文档、知识块、AI 会话必须拆表治理**
3. **Controller 轻、Service 重、Mapper 纯访问**
4. **AI 必须严格基于权限过滤后的知识库内容**
5. **工具能力必须通过白名单 + 审计 + 参数校验受控执行**
6. **文档内容建议结构化存储，不建议只存 HTML**
7. **V1 不要过度设计协同编辑，先把单人知识闭环做完整**

24.2 对你这个项目最重要的三件事

1. 先把“项目—文件—文档—搜索”主链路做扎实
2. 再把“AI 严格基于知识库回答并给引用”做可信
3. 最后再逐步开放工具能力与 MCP 扩展

---

25. 后续可继续补充的配套文档

后续建议继续拆分输出以下文档： 1. 《数据库详细设计文档》 2. 《OpenAPI 接口文档》 3. 《前端页面与路由设计文档》 4. 《后端模块任务拆解文档》 5. 《AI 微服务与 RAG 详细设计文档》 6. 《权限模型与安全设计文档》 7. 《部署架构与运维手册》

---

26. 本版交付结语

本版为 Erise-AI 的企业级开发设计文档基线版，适合作为项目启动、研发分工、数据库建模、接口约定和 AI 能力设计的统一依据。后续建议在此基础上继续细化成可直接开发的数据库、接口、任务拆解与部署文档。